



Base de datos II

Grupo 26

Apellido y Nombre	Legajo
SEBASTIÁN YANNI	32.398
BIANCHI MAXIMILIANO	32.538
REARTE BECERRA MARCELO ADRIÁN	32.338

TinyDesk

Descripción general

Para el desarrollo del trabajo práctico se decidió continuar y evolucionar un proyecto realizado previamente llamado **TinyDesk**.

TinyDesk fue desarrollado originalmente durante el segundo cuatrimestre como una aplicación de consola en **C++**, orientada a la gestión básica de proyectos, tareas y tickets. Para este nuevo trabajo práctico, la propuesta consiste en realizar una **migración** llevándolo a una aplicación incorporando principalmente una base de datos relacional en **SQL Server**.

El sistema está pensado como una aplicación similar a **Jira**, **Trello** o **ClickUp**, donde los usuarios podrán administrar proyectos, dividirlos en sprints, crear tickets, asignarlos a usuarios, controlar prioridades y modificar estados de avance.

Además, el proyecto nos permitiría mostrar la capacidad de adaptar un sistema existente a una nueva tecnología, migrando desde una solución de consola en **C++** hasta manejar datos, tablas e información **SQLServer**. Esto resulta útil tanto para la materia como para nuestro portfolio personal y profesional.

[DER DRAW](#)

[GitHub](#)

Procedimientos almacenados

Los procedimientos almacenados se utilizarán para centralizar las operaciones principales del sistema. De esta manera, las acciones más importantes no dependerán directamente de consultas sueltas, sino que estarán definidas dentro de la base de datos.

Se propone implementar los siguientes procedimientos:

Crear proyecto

Se utilizará un procedimiento almacenado para crear nuevos proyectos. Este recibirá los datos principales del proyecto, como nombre, descripción, fecha de inicio, fecha de fin y estado inicial. El campo Activo quedará con valor por defecto en 1, indicando que el proyecto se encuentra activo.

Crear sprint

Se utilizará un procedimiento almacenado para crear sprints asociados a un proyecto. Cada sprint tendrá una fecha de inicio, una fecha de fin, un proyecto relacionado, un estado y un área responsable.

Crear ticket

Se utilizará un procedimiento almacenado para registrar tickets dentro de un sprint. El ticket incluirá descripción, fecha de inicio, fecha de fin, prioridad, usuario asignado, estado y sprint correspondiente.

Cambiar estado de ticket

Este procedimiento permitirá modificar el estado de un ticket, por ejemplo, pasarlo de pendiente a en progreso o de en progreso a finalizado. Es una operación fundamental en un sistema de gestión de tareas.

Reasignar ticket

Este procedimiento permitirá cambiar el usuario responsable de un ticket. Esto es útil cuando una tarea debe ser transferida a otro integrante del equipo.

Cerrar ticket

Este procedimiento permitirá finalizar un ticket. Al cerrar un ticket, se podrá actualizar su estado a finalizado y registrar la fecha de finalización.

Vistas

Las vistas se utilizarán para simplificar las consultas y mostrar información más clara al usuario final. En lugar de consultar varias tablas con JOIN cada vez, las vistas permitirán obtener información ya relacionada.

Se propone implementar las siguientes vistas:

Vista de usuarios detallados

Esta vista mostrará la información principal de los usuarios junto con su rol y área correspondiente. Permitirá consultar fácilmente qué usuarios existen en el sistema, a qué área pertenecen y qué permisos tienen.

Ejemplo:

NombreUsuario,
Nombre,
Apellido,
Rol,
PermisoEscritura,
Area,
Activo

Vista de proyectos detallados

Esta vista mostrará los proyectos junto con su estado. Permitirá listar los proyectos de manera más clara, sin necesidad de consultar manualmente la tabla ESTADO.

Ejemplo:

Proyecto,
Descripcion,

FechaInicio,
FechaFin,
Activo,
Estado

Vista de sprints detallados

Esta vista mostrará los sprints junto con el proyecto al que pertenecen, su estado y el área responsable.

Ejemplo:

SprintId,
FechaInicio,
FechaFin,
Proyecto,
Estado,
Area,
Activo

Vista de tickets detallados

Esta será una de las vistas principales del sistema.

Permitirá ver los tickets

junto con su prioridad, usuario asignado, estado, sprint y proyecto relacionado.

Ejemplo:

TicketId,
Descripcion,
FechaInicio,
FechaFin,

Tiempo hasta finalizarlo,
Dias Abierto,
Situacion Actual,
Prioridad,
Estado,
UsuarioAsignado,
Usuario Activo,
Area usuario,
Rol usuario,
Sprint numero,
Sprint Activo,
Proyecto nombre,
Proyecto descripción,
Proyecto Fecha Fin,
Proyecto Activo

Vista de tickets pendientes

Esta vista mostrará únicamente los tickets que todavía no fueron finalizados.

Será útil para representar un tablero de trabajo donde se visualicen las tareas pendientes o en curso.

Vista de proyectos activos

Esta vista mostrará solamente los proyectos activos, es decir, aquellos cuyo campo Activo tenga valor 1.

Vista de usuarios activos

Esta vista mostrará únicamente los usuarios activos del sistema. Esto permite evitar mostrar usuarios dados de baja en las pantallas principales de la aplicación.

Triggers

Los triggers se utilizarán para validar reglas de negocio que deben cumplirse automáticamente al insertar o actualizar información en la base de datos.

Se propone implementar los siguientes triggers:

Impedir modificación de fecha de inicio de proyecto

Este trigger impedirá que se modifique la fecha de inicio de un proyecto una vez creado. La fecha de inicio se considera un dato importante del proyecto y debe conservarse para mantener la trazabilidad.

Validar fecha de inicio de sprint

Este trigger controlará que, al crear un sprint, la fecha de inicio no sea anterior a la fecha actual.

Impedir modificación de fecha de inicio de sprint

Este trigger impedirá que la fecha de inicio de un sprint sea modificada luego de su creación, ya que un sprint representa un período de trabajo definido.

Validar fecha de inicio de ticket

Este trigger controlará que, al crear un ticket, la fecha de inicio no sea anterior a la fecha actual.

Baja lógica de proyecto

Se utilizará un trigger para evitar la eliminación física de proyectos. En caso de intentar eliminar un proyecto, el sistema deberá impedirlo o convertir esa acción en una baja lógica, modificando el campo Activo a 0.

Baja lógica de sprint

Se utilizará un trigger para evitar la eliminación física de sprints. Para esto, será necesario agregar el campo Activo a la tabla SPRINT. En lugar de borrar el sprint, se marcará como inactivo.

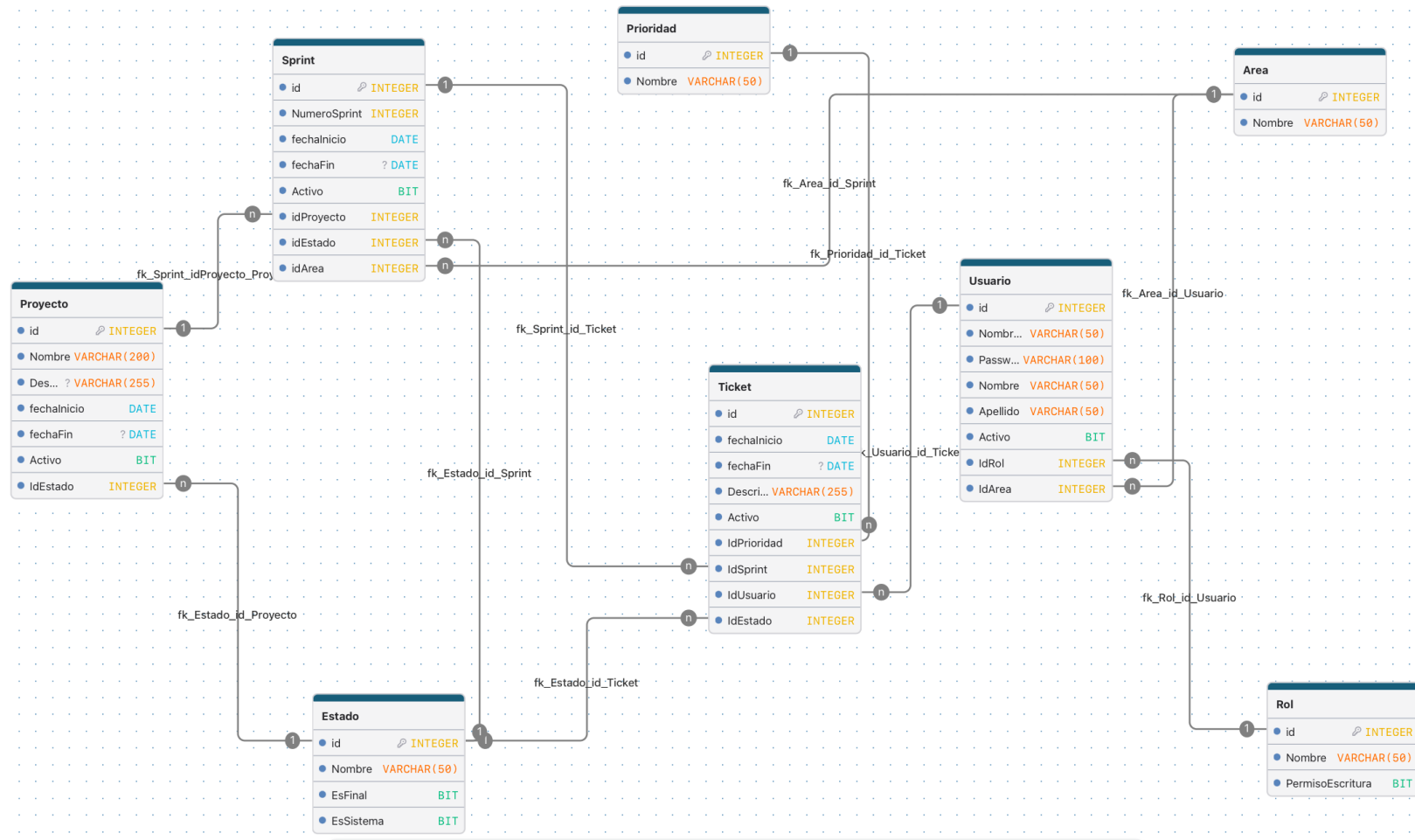
Baja lógica de ticket

Se utilizará un trigger para evitar la eliminación física de tickets. Para esto, será necesario agregar el campo Activo a la tabla TICKET. En lugar de borrar el ticket, se marcará como inactivo.

Baja lógica de usuario

Se utilizará un trigger para evitar la eliminación física de usuarios. En caso de intentar eliminar un usuario, se modificará su campo Activo a 0, conservando sus relaciones con tickets y evitando problemas de integridad referencial.

DER:



URL DER:

[DER DRAW](#)